

typed-dsa showcase

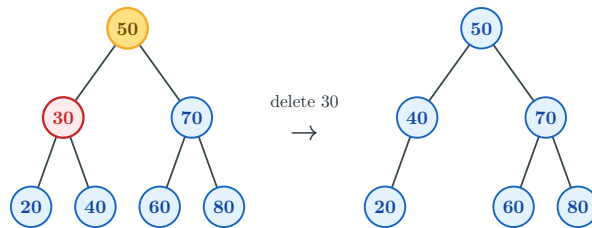
A deliberately over-styled pass through the package's main diagram families.

Trees and AVL transitions

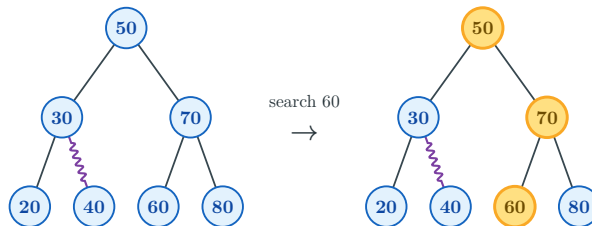
```
#let a = avl(50, 30, 70, 20, 40,  
  style: your-style)  
#(a.insert)(35, rebalance: (enabled:  
  true, all-steps: true)).diagram
```



```
#let b = bst(50, 30, 70, 20, 40, 60, 80,  
  style: your-style)  
#(b.delete)(30).diagram
```

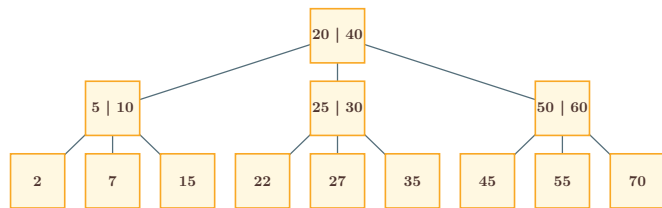


```
#let c = bst(50, 30, 70, 20, 40, 60, 80,  
  style: your-style)  
#(c.search)(60).diagram
```

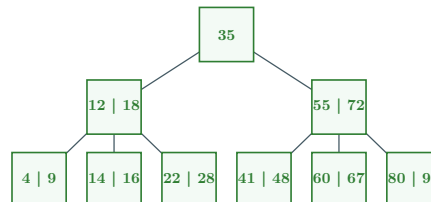


Multiway trees and table-like structures

```
#tree(  
  node([20 | 40], children: (  
    node([5 | 10], children: (...)),  
    node([25 | 30], children: (...)),  
    node([50 | 60], children: (...)),  
  )),  
  style: your-style,  
)
```



```
#tree(  
  node([35], children: (  
    node([12 | 18], children: (...)),  
    node([55 | 72], children: (...)),  
  )),  
  style: your-style,  
)
```



Arrays and matrices

```
#array-view(  
  8, 13, 21, 34, 55, 89, 144, 233, 377,  
  style: your-style + (indices: (enabled:  
true)),  
  cell-customizations: ((4, pivot-style),  
(5, pivot-style)),  
)
```

8	13	21	34	55	89	144	233	377
0	1	2	3	4	5	6	7	8

```
#matrix(  
  ((0, 4, 8, 12, 16), ...),  
  row-labels: ([A], [B], [C], [D], [E]),  
  column-labels: ([A], [B], [C], [D],  
[E]),  
  style: your-style,  
)
```

	A	B	C	D	E
A	0	4	8	12	16
B	3	0	5	9	13
C	7	2	0	6	10
D	11	6	1	0	4
E	15	10	5	2	0

Sorting algorithms: pseudocode and traces

Each trace uses a small array and shows every generated step.

Bubble sort

BUBBLE SORT():

```

1  for  $p \leftarrow 0$  to  $n - 2$ :
2      for  $j \leftarrow 0$  to  $n - p - 2$ :
3          if  $A_j > A_{j+1}$ :
4              swap  $A_j$  and  $A_{j+1}$ 

```

Step 1
start

3	1	2
0	1	2

Step 2
compare 3 and 1

j	$j+1$	
↓	↓	
3	1	2
0	1	2

Step 3
swap 3 and 1

j	$j+1$	
↓	↓	
1	3	2
0	1	2

Step 4
compare 3 and 2

	j	$j+1$
	↓	↓
1	3	2
0	1	2

Step 5
swap 3 and 2

	j	$j+1$
	↓	↓
1	2	3
0	1	2

Step 6
3 settled

1	2	3
0	1	2

Step 7
compare 1 and 2

j	$j+1$	
↓	↓	
1	2	3
0	1	2

Step 8
2 settled

1	2	3
0	1	2

Step 9
sorted

1	2	3
0	1	2

Insertion sort

INSERTION SORT($\cdot$):

```
1  for  $i \leftarrow 1$  to  $n - 1$ :  
2     $k \leftarrow A[i]$   
3     $j \leftarrow i$   
4    while  $j > 0$  and  $A[j] <$   
       $A[j - 1]$ :  
5      swap  $A[j]$  and  $A[j - 1]$   
6       $j \leftarrow j - 1$ 
```

Step 1
start

3	1	2
0	1	2

Step 2
compare 3 and 1

$j-1$	j	
↓	↓	
3	1	2
0	1	2

Step 3
swap 3 and 1

$j-1$	j	
↓	↓	
1	3	2
0	1	2

Step 4
compare 3 and 2

	$j-1$	j
	↓	↓
1	3	2
0	1	2

Step 5
swap 3 and 2

	$j-1$	j
	↓	↓
1	2	3
0	1	2

Step 6
compare 1 and 2

$j-1$	j	
↓	↓	
1	2	3
0	1	2

Step 7
sorted

1	2	3
0	1	2

Selection sort

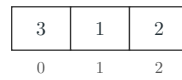
SELECTION SORT($\cdot$):

```

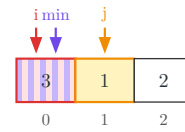
1  for  $i \leftarrow 0$  to  $n - 1$ :
2       $m \leftarrow i$ 
3      for  $j \leftarrow i + 1$  to  $n - 1$ :
4          if  $A[j] < A[m]$ :
5               $m \leftarrow j$ 
6      swap  $A[i]$  and  $A[m]$ 

```

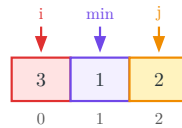
Step 1
start



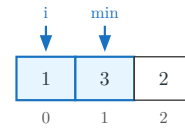
Step 2
position 0, minimum 0, item 1



Step 3
position 0, minimum 1, item 2



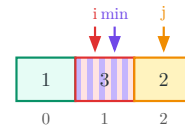
Step 4
swap 3 and 1



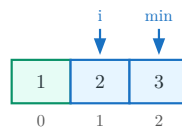
Step 5
1 settled



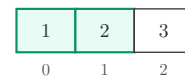
Step 6
position 1, minimum 1, item 2



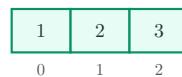
Step 7
swap 3 and 2



Step 8
2 settled



Step 9
sorted



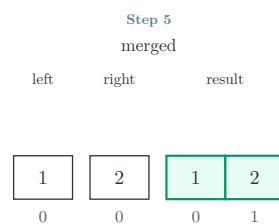
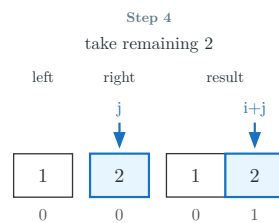
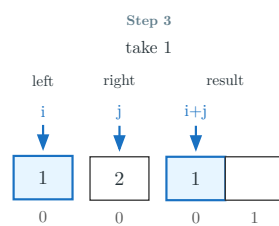
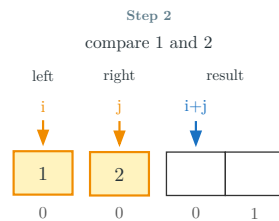
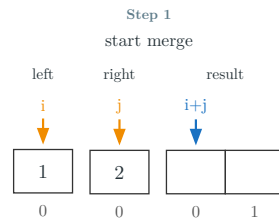
Merge operation

MERGE():

```

1   $i \leftarrow 0; j \leftarrow 0; M \leftarrow ()$ 
2  while  $i < |L|$  and  $j < |R|$ :
3      if  $L_i \leq R_j$ :
4          append  $L_i$  to  $M$ ;  $i \leftarrow i + 1$ 
5      else:
6          append  $R_j$  to  $M$ ;  $j \leftarrow j + 1$ 
7  append remaining  $L$  and  $R$  to  $M$ 
8  return  $M$ 

```



Quick sort: last-pivot partition

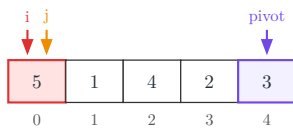
LAST-PIVOT PARTITION():

```

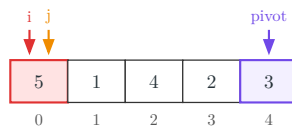
1   $p \leftarrow A_{n-1}$ 
2   $i \leftarrow 0$ 
3  for  $j \leftarrow 0$  to  $n - 2$ :
4      if  $A_j \leq p$ :
5          swap  $A_i$  and  $A_j$ 
6           $i \leftarrow i + 1$ 
7  swap  $A_i$  and  $A_{n-1}$ 

```

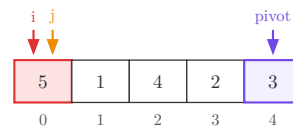
Step 1
start
pivot: 3 at 4 i: 0 j: 0



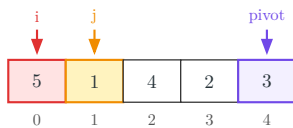
Step 2
select last pivot 3
pivot: 3 at 4 i: 0 j: 0



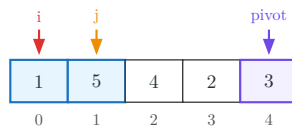
Step 3
compare 5 and pivot 3
pivot: 3 at 4 i: 0 j: 0



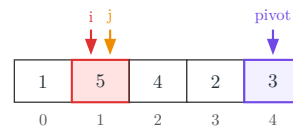
Step 4
compare 1 and pivot 3
pivot: 3 at 4 i: 0 j: 1



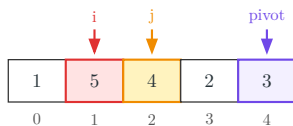
Step 5
swap 5 and 1
pivot: 3 at 4 i: 0 j: 1



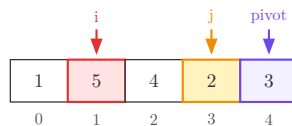
Step 6
advance i to 1
pivot: 3 at 4 i: 1 j: 1



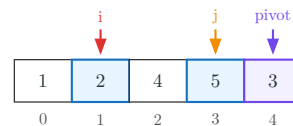
Step 7
compare 4 and pivot 3
pivot: 3 at 4 i: 1 j: 2



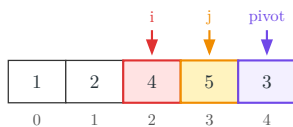
Step 8
compare 2 and pivot 3
pivot: 3 at 4 i: 1 j: 3



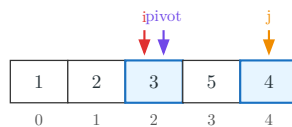
Step 9
swap 5 and 2
pivot: 3 at 4 i: 1 j: 3



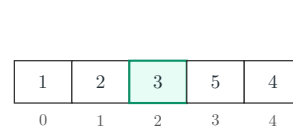
Step 10
advance i to 2
pivot: 3 at 4 i: 2 j: 3



Step 11
place pivot 3
pivot: 3 at 2 i: 2 j: 4

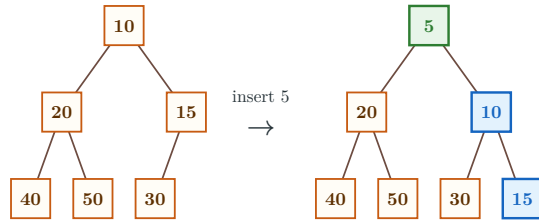


Step 12
partitioned
pivot: 3 at 2 i: 2 j: 4

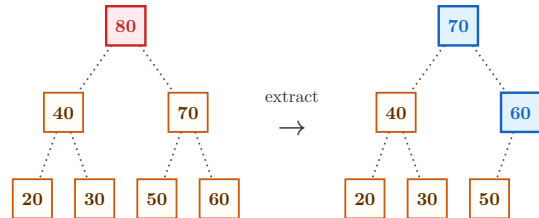


Heaps with sift paths

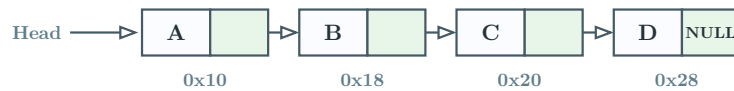
```
#let h = min-heap(10, 20, 15, 40, 50, 30,
style: your-style)
#(h.insert)(5).diagram
```



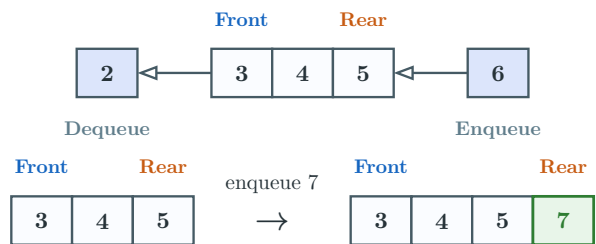
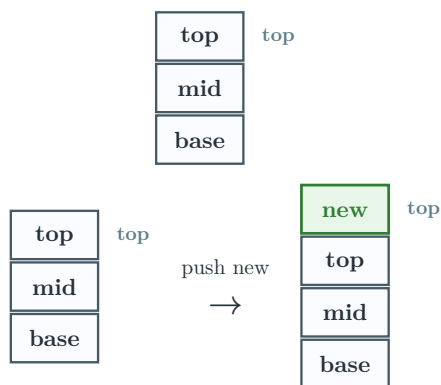
```
#let h = max-heap(50, 30, 70, 20, 40, 60,
80,
style: your-style)
#(h.extract)().diagram
```



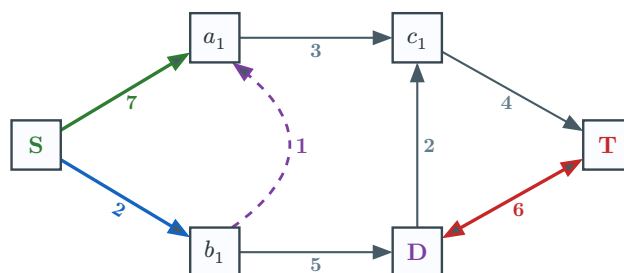
Linked structures



Stack and queue workflows



Weighted graph with custom edge labels



Hand-composed rotation schematic

